

P1714: NTTP are incomplete without float, double, and long double! (Revision 0)

Revision history

- 2019-06-17: R0: Proposal to allow float, double, and long double template parameters.

Introduction.

For decades, template parameters could be either types or constants of certain trivial types: integers, enums, pointers, etc. Notably absent from this list were floating-point values. Recently, the adoption of P0732 has allowed constants of class type to be used as template parameters. Furthermore, P0476 allows us to perform constexpr bit-casting of floating-point values. And in the decades since floating-point types were banned from use as template parameters, compile-time computation of floating-point values has advanced dramatically.

This paper, P1714, proposes to include floating-point values into the list of acceptable template parameters.

I. Motivation and Scope.

Consider the `pow()` function. The most general implementation uses `log` and `exp`:

```
double pow(double base, double exponent) {
    return exp(log(base) * exponent);
}
```

This has several disadvantages, one being that if `exponent` is an integer, the exactness available through multiplication isn't achieved due to round-off error in `exp` and `log`. So we end up in the unfortunate situation that raising an integer to an integral power sometimes produces a result that is very close to, but not equal to, an integer. Similarly, often a number is raised to the power $1/2$ in an attempt to obtain a square root, esp. by programmers from other languages who are unaware that the standard library offers an extremely accurate square-root instruction.

But suppose we could specify the exponent:

```
template<double exponent>
double pow(double base);
```

The default implementation of such a function could use the `log/exp` solution, while the code could be specialized for common integer powers and binary fractions ($1/2$, $1/4$) to produce far more accurate results - and to produce them faster. There's just one problem: the floating-point exponent is not allowed as a template parameter.

With the new facilities of C++20, we can work around this problem: (Working demo at [Compiler Explorer](#))

```
template<typename T> struct AsTemplateArg {
    std::array<char, sizeof(T)> buffer = {};
    constexpr AsTemplateArg(const std::array<char, sizeof(T)> buf) : buffer(buf) {}
    constexpr AsTemplateArg(T t) : AsTemplateArg(std::bit_cast<std::array<char, sizeof(T)>>(t)) {}
    constexpr operator T() const { return std::bit_cast<T>(this->buffer); }
};

template<AsTemplateArg<double> exponent>
double pow(double base) {
    return exp(log(base) * double{exponent});
}

template<>
```

```
double pow<ATemplateArg<double>{1.0}>(double base) {
    return base;
}
```

But why? Let's just let the compiler do what it can do very easily, rather than force the use of a bunch of bit-cast boilerplate.

Impact on the Standard

Portions of the standard which currently prohibit use of floating-point constants as template parameters shall be removed.

Proposed Wording

Note: All changes are relative to the 2019-06-13 working draft of C++20.

Modify 13.1 [temp.param] :

```
A non-type template-parameter shall not be declared to have floating-point or void type. [Example:
template<double d void romeo > class X;    // error
template<double void* pd> class Y;    // OK
template<double& rd> class Z;    // OK
–end example ]
```

Alternatives Considered

Originally the suggested design was to decompose a floating-point type into sign, exponent, and mantissa, and then use the existing P0732 wording to allow that triplet of values to represent the floating-point constant in question. This was changed because:

- 1) It's more work for the compiler. All compilers must already know how to represent floating-point values in bit form for their target architecture, in case the user declares a global floating-point value with an initial value. And that bitwise representation satisfies P0732's requirements for template parameter, no decomposition needed.
- 2) Such a decomposition does not distinguish between positive zero and negative zero, which would prohibit the implementation of a function such as pow, which distinguishes between positive and negative zero. Even printf is defined to treat +0.0 and -0.0 differently.
- 3) Such a decomposition proves difficult for INF and NaN values.

Downsides

Using bit-level equality rather than the type's underlying operator== means that if you specialize a template that uses float/double parameters, using 0.0 as your specialization, then your specialization will not impact code that passes -0.0 as a parameter.

There is a related impact with NaNs; attempting to specialize such a template using a value of NAN or -NAN will only specialize those two NaNs, rather than the full range of NaNs that exist. Nevertheless, this is not expected to be an issue; if a user wants to have different template behavior for NaNs, it's a simple matter of adding:

```
if constexpr(!(float_param == float_param)) {
    // Handle NaN
}
```

References

- JF Bastien, [Bit-casting object representations](#)
- Jeff Snyder, [Class Types in Non-Type Template Parameters](#)